Deploying the MLOps inner loop

In Databricks, the MLOps inner loop uses a variety of tools within the DI platform that we've already touched upon throughout this book, such as MLflow, Feature Engineering with Unity Catalog, and Delta. This chapter will highlight how you can leverage them together to facilitate MLOps from one place. MLOps is covered in even more depth by Databricks' ebook, *The Big Book of MLOps*, which we highly recommend if you wish to learn more about the guiding principles and design decisions when architecting your own MLOps solution. We use **GitHub** to help facilitate DevOps and code reproducibility. For the DataOps portion, we use **Unity Catalog** and **Delta**. These tools help us track the versions of data and the code associated with the features created. This is the data reproducibility piece of DataOps. We use Delta time travel to query data from previous versions of the same table in the short term. For long term reproducibility, we recommend saving the training dataset as a Delta table and logging the table path with the `mlflow.log_input()` method. The upstream sources of the feature tables we create are automatically tracked by UC lineage.

Registering a model

Let's jump into model registries and their role in productionalization. A model registry is a centralized model store that helps to manage the entire model lifecycle, including versioning or aliasing, CI/CD integration, webhooks, and notifications. In Databricks, the **Unity Catalog Registry** (**UC Registry**) extends the governance features of Unity Catalog to the model registries, including centralized access control, auditing, lineage, and model discovery across workspaces. The UC Registry is a centralized repository for your models and their chronological lineage. This means that the experiment and experiment run that created each respective model is linked to the respective code and data source. Once you are satisfied with your machine learning model, the first thing to do is register it in Databricks under your central model registry. A registered model is a logical grouping of a model's version history. The UC Registry tracks different model versions, including each version's training data, hyperparameters, and evaluation metrics. A model rarely, if ever, has only one version. In addition to experimentation of model types and hyperparameter values, there are also cases where we want to experiment with different versions of a model. We can refer to these different model versions using model aliases. For example, deploying different versions simultaneously can be helpful for A/B testing; we'll cover this in more detail in the *Model Inference* section below. With the UC Registry, you can easily create and manage multiple model aliases, making it easier to track changes, compare performance, and revert to earlier versions if needed.

Collaborative Development

The UC Registry provides a collaborative model development environment, enabling teams to share and review models. This collaboration is also tracked, allowing multiple teams or leads to stay abreast of the model development process. Team or project leads can also require documentation before allowing a model to continue through the lifecycle. We find it easier to add snippets of documentation throughout the project rather than trying to remember everything and take the time to write it all down afterward.